

Butterfly Counting over Bipartite Graphs with Local Differential Privacy

Yizhang He¹ Kai Wang² Wenjie Zhang¹ Xuemin Lin² Wei Ni³ Ying Zhang⁴

¹University of New South Wales

²Antai College of Economics and Management, Shanghai Jiao Tong University

³Data61, CSIRO, ⁴University of Technology Sydney

¹{yizhang.he, wenjie.zhang}@unsw.edu.au ²{w.kai, xuemin.lin}@sjtu.edu.cn

³wei.ni@data61.csiro.au ⁴ying.zhang@uts.edu.au

Abstract— Butterfly counting on bipartite graphs has gained increasing attention in past decades. Inevitably, butterfly counts can reveal the presence of certain edges, posing a privacy risk in real applications. Edge local differential privacy (edge LDP), which requires each vertex to perturb its neighbors locally, has been applied to protect edge privacy in graphs. This paper, for the first time, investigates butterfly counting on bipartite graphs with edge LDP. Although a straightforward approach that allows each vertex to perturb its incident edges locally to construct a noisy graph and perform butterfly counting preserves edge LDP, it often results in severe over-counting and significant bias since the resulting noisy graph is generally much denser than the input graph. To obtain unbiased butterfly counts, we propose a multiple-round interaction algorithm to allow the vertices to download the noisy graph and compute local motif counts. Moreover, to avoid adding substantial noise to satisfy edge LDP, we further propose the Download-free Butterfly Estimation (DBE) algorithm, which captures motif transformation probabilities and relies on motif counts from the noisy graph to yield unbiased butterfly estimates. DBE significantly enhances accuracy via reduced communication between vertices and the data curator. Extensive experiments on 14 datasets validate the effectiveness and efficiency of our proposed techniques.

I. INTRODUCTION

Bipartite graphs arise as a natural fit for modeling the connections between two sets of entities. Examples of real-world bipartite networks include user-item networks in E-commerce [1], people-location networks in contact tracing [2], and user-page networks in social analysis [3]. On bipartite graphs, a butterfly (i.e., a (2×2) -biclique) is the smallest cohesive unit, which is analogous to triangles in general graphs [4], [5], [6], [7], [8], [9], [10], [11], [12]. Given a bipartite graph G , butterfly counting [13] aims to count the number of butterflies in G , which benefits many applications including fraud detection [8], [12], recommendation system [14], bipartite clustering coefficient computation [15], [16], link prediction [17], web traffic monitoring [18], and maximum biclique search [19]. However, releasing the exact butterfly counts on bipartite graphs can pose a threat to individuals' privacy. For example, consider the user-item networks in Fig. 1(a), where each edge indicates that a user has bought an item. In the graph G_1 , there are 3 users and 100 items, and u_3 has bought all these items. When the user u_2 makes another purchase (v_2), an additional edge (u_2, v_2) is added to G_1 , resulting in a graph G_2 . Suppose that u_1 and u_3 collude

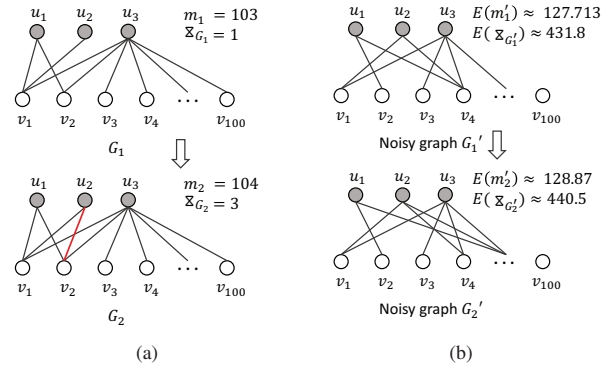


Fig. 1: Two user-item networks and their corresponding noisy graphs constructed via randomized responses.

and try to infer what u_2 has bought. If the butterfly counts for G_1 and G_2 are released, u_1 and u_3 will be able to observe the butterfly count changes from 1 to 3 and infer that u_2 has bought the item v_2 , leading to privacy breaches.

Motivated by the above observations, we study the problem of privacy-preserving butterfly counting on bipartite graphs. Specifically, given a bipartite graph G , we aim to compute the butterfly count of G under *edge local differential privacy* (edge LDP) [20], [21], [22]. A random algorithm satisfies edge LDP if, for any two vertices whose neighbors differ by one edge (u, v) , the probabilities of observing the algorithmic outputs from these two vertices are indistinguishable within a factor of e^ϵ . This provides deniability for the existence of the edge (u, v) . Here, ϵ is the privacy budget, gauging the amount of permissible privacy loss. This parameter can be adjusted by the client to modulate the tradeoff between the level of privacy and data utility. Edge LDP ensures robust privacy protection by requiring each vertex to perturb its local data before transmitting it to the data curator. This approach does not assume that the data curator must be “benign” and trusted, unlike existing works under *Central Differential Privacy* (CDP) [23], [24], [25], [26], [27], [28]. Compared to CDP, where privacy is jeopardized for all users if the data curator storing the whole graph is hacked [29], [30], Edge

LDP offers a more resilient privacy protection mechanism. In addition, studying butterfly counting under edge LDP is the first step in addressing other graph computing tasks under differential privacy, such as k-bitruss computation [8], [31], and (p, q) -biclique search and counting [32].

Challenges. Edge LDP has been applied to triangle counting, k -core decomposition, densest subgraph discovery, and degree ordering [29], [33], [34] on unipartite graphs. Some of the existing approaches [29], [33] adopt *Randomized Response* [35] to provide edge LDP, which perturbs each vertex's edges and uses these noisy edges to construct a noisy graph. We can also apply randomized responses to a bipartite graph and return the butterfly count on the noisy graph. This *Naive* approach leads to severe over-counting and large bias because the noisy graph is generally much denser than the original graph. For example, consider the bipartite graph G_1 in Fig. 1(b) with $\mathbf{x}_G = 1$. When the privacy budget $\epsilon = 1$, the flipping probability is approximately 0.269. The resulting noisy graph G'_1 is expected to have 431.8 butterflies on average from 1000 runs. As for G_2 , its noisy graph G'_2 is expected to have approximately 440.5 butterflies, significantly exceeding the actual count. In this paper, we aim to design accurate and efficient butterfly counting algorithms on bipartite graphs with edge LDP and face these main challenges.

- **Challenge 1:** Given an input bipartite graph G with a fill rate of $d = \frac{m}{n_1 \times n_2}$ (m, n_1 , and n_2 are the number of edges, upper vertices, and lower vertices in G), the noisy graph G' due to randomized responses has an expected fill rate of $d(1 - 2p) + p$ (where $p = \frac{1}{1 + e^\epsilon}$), which is much denser and has many more butterflies than G . Thus, it is a challenge to estimate the butterfly count without bias based on the noisy graph.

- **Challenge 2:** Due to the bipartite graph structure, a vertex in a butterfly cannot reach other vertices of the butterfly by visiting its neighbors. For instance, in Fig. 1, u_1 cannot observe edges (u_2, v_1) and (u_2, v_2) even though they form a butterfly with u_1 . In the meantime, on unipartite graphs, a vertex in a triangle can reach the other two vertices by visiting its neighbors. Thus, it is challenging in butterfly counting to send useful graph statistics to the data curator under edge LDP.

Our approaches. To address Challenge 1, we propose the *MultiR* algorithm with three rounds of interaction between vertices and the data curator. In the first round, *MultiR* allows each vertex to perturb its degree and obtains the estimates of the vertex degrees. The second round involves constructing a noisy graph by applying randomized responses to the neighbors of each vertex. In the third round, each vertex downloads the noisy graph and counts the “local butterflies” and “local caterpillars” (a caterpillar is a motif that is one edge away from forming a butterfly) with two edges in the input graph and other edges in the noisy graph. To satisfy edge LDP, we apply the Laplacian mechanism [36] to add noise to the local butterfly and caterpillar counts. These counts are adjusted based on the global sensitivity, which denotes the largest difference observed in butterfly and caterpillar counts between two vertices with neighbors differing by a single edge. At last, these counts are aggregated for unbiased butterfly

count estimates.

To tackle Challenge 2, we employ a non-interactive approach and present a *Download-free Butterfly Estimation* (DBE) algorithm that provides unbiased and precise butterfly estimates. DBE relies solely on randomized responses to satisfy edge LDP and directly uses the noisy graph for estimates of butterfly counts. To achieve this, DBE examines how motifs with two upper and lower vertices on the input graph transform into those on the noisy graph, as the edges are flipped during randomized responses. Based on this, DBE establishes the connections between the expected values of motif counts on the noisy graph and the butterfly count on the input graph, resulting in a closed-form expression for butterfly count estimation. Unlike *MultiR*, DBE utilizes only the motif counts on the noisy graph, thus avoiding the “curse of high global sensitivity” by minimizing communication between vertices and the data curator, leading to significant accuracy improvement.

Contributions. Here we summarize our main contributions.

- To the best of our knowledge, we are the first to investigate efficient and accurate butterfly counting on bipartite graphs under edge LDP.
- We introduce the *MultiR* algorithm, which employs multiple rounds of interactions between vertices and the data curator to address the “dense noisy graph” issue. By statistically aggregating local butterfly and caterpillar counts, *MultiR* produces unbiased estimates for butterfly count.
- We propose the DBE algorithm to improve data utility. By adopting a download-free framework, DBE minimizes excessive communication between vertices and the data curator, addressing the challenges of high global sensitivity. In addition, DBE accurately models motif transformation and derives a statistically sound formula for unbiased butterfly count estimation.
- We conduct extensive experiments on 14 real-world datasets to evaluate the proposed algorithms. In terms of effectiveness, DBE consistently yields significantly smaller relative errors than *Naive* and *MultiR* across all datasets. In addition, DBE incurs a comparable time cost to *Naive* and minimal communication cost among the three butterfly counting algorithms under edge LDP.

II. PROBLEM DEFINITION

TABLE I: Summary of Notations

Notation	Definition
G	a bipartite graph
$\mathcal{A}$	the adjacency matrix
$\mathcal{A}_u$	the neighbor list of the vertex u
$N(u, G)$	the neighbors of u in G
$\deg(u, G)$	the degree of u in G
ϵ	a privacy budget
G'	a noisy bipartite graph
$\mathbf{x}_G$	the number of butterflies in G
Δf	the global sensitivity of a function f

In this section, we first present important definitions and notations about butterfly and differential privacy over bipartite graphs. Then, we present the problem statement.

A. Butterflies on bipartite graphs

We consider an unweighted bipartite graph $G(V = (U, L), E)$. $V = U \cup L$ denotes the set of vertices, where U and L represent the upper and lower layer, respectively. The vertices in U and L are called upper vertices and lower vertices. $E \subseteq U \times L$ denotes the set of edges. We use $n_1 = |U(G)|$ and $n_2 = |L(G)|$ to denote the number of upper and lower vertices, respectively, and $m = |E|$ to represent the number of edges in G . The adjacency matrix $\mathcal{A}$ for G is of dimensions $n \times n$, where $\mathcal{A}[u, v] = 1$ if there exists an edge between the vertices u and v and 0 otherwise. The u -th row of $\mathcal{A}$ (including both “1” and “0”) is the *neighbor list* of u , denoted by $\mathcal{A}_u$. In addition, the set of neighbors of a vertex u in G is denoted by $N(u, G)$, and its degree is denoted by $\deg(u, G) = |N(u, G)|$. We use $d_{\max}(U)$ and $d_{\max}(L)$ to represent the maximum degree among the upper vertices and lower vertices, respectively.

In this paper, we are interested in butterflies on bipartite graphs. Butterflies are considered as the smallest units of cohesiveness and the analogs of triangles [4], [5], [6], [8]. Caterpillars are closely related motifs on bipartite graphs, which are one edge away from forming butterflies.

Definition 1. Butterfly. In a bipartite graph G , given vertices $u, w \in U(G)$ and $v, x \in L(G)$, a butterfly is the complete subgraph induced by u, v, w, x , which means both u and w are connected to v and x by edges. The total number of butterflies in G is denoted as $\mathfrak{x}_G$.

Definition 2. Caterpillar. Given a bipartite graph G , a caterpillar is a 3-path on G , which consists of four vertices and three edges.

B. Differential privacy on bipartite graphs

Classic differential privacy (DP) assumes a central model, where a central data curator manages a dataset D [36]. Two datasets D and D' are considered neighboring datasets if they differ by one data record. For any neighboring datasets, DP ensures that the outputs of a query function on these datasets are hard to distinguish. When DP under the central model is applied to graphs to protect edge privacy, the neighboring datasets are two graphs that differ by one edge. While this central model offers better data utility, the assumption that the data curator storing the entire graph can be trusted is often unattainable. Thus, we adopt the widely-used ϵ -edge local differential privacy (edge LDP), which allows each vertex to perturb its data locally before sending it to the data curator [37], [21], [22]. Under edge LDP, the neighboring datasets are two neighbor lists that differ by one bit.

Definition 3. ϵ -edge local differential privacy. Let G be a bipartite graph and $\epsilon > 0$. For each vertex $u \in V(G)$, let R_u with domain $\{0, 1\}^n$ be a randomized algorithm of vertex u .

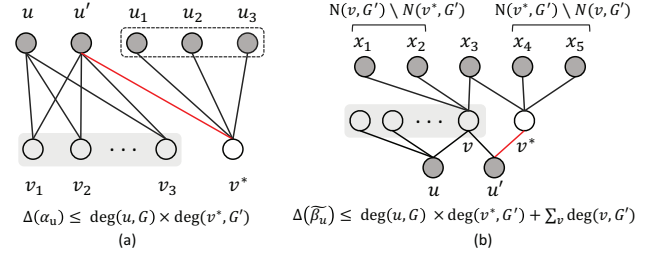


Fig. 2: Proof illustration for Lemma 2. The red line indicates the edge between u and v^* .

R_u provides ϵ -edge LDP if for any two neighbor lists $\mathcal{A}_u, \mathcal{A}'_u$ that differ in one bit and any $S \subseteq \text{Range}(R_i)$:

$$\Pr(R_u(\mathcal{A}_u) \in S) \leq e^\epsilon \Pr(R_u(\mathcal{A}'_u) \in S)$$

Here ϵ is called the privacy budget.

Note that ϵ -edge LDP ensures that if two vertices have neighbor lists that differ by one bit, they cannot be reliably distinguished based on the outputs from the randomized algorithms. Consider t edge LDP algorithms $Z_1, Z_2, \dots, Z_t$, whose privacy budgets are denoted by $\epsilon_1, \epsilon_2, \dots, \epsilon_t$, respectively. We have the following property.

Theorem 1. Sequential Composition [30]. The composite algorithm obtained by sequentially applying $Z_1, Z_2, \dots, Z_t$ on the bipartite graph G provides $\sum_{i=1}^t \epsilon_i$ -differential privacy.

Warner’s randomized response. The easiest way to provide ϵ -edge LDP is via randomized responses (hereafter denoted by RR), which was originally proposed as a survey technique to allow confidential answers to sensitive issues (e.g. criminal or sexual activities) [35]. In essence, the participants are asked to answer questions truthfully with probability. This has been applied to graphs to satisfy edge local differential privacy [20], [29]. Specifically, given a privacy budget ϵ , each entry $x \in \{0, 1\}$ of a neighbor list is perturbed with probability $\frac{1}{1+e^\epsilon}$:

$$RR(x) = \begin{cases} 1 - x & \text{with probability } \frac{1}{1+e^\epsilon} \\ x & \text{with probability } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$$

The state-of-the-art methods on general graphs perturb each entry in the lower triangle of the adjacency matrix [29], [33]. However, to ensure the resulting noisy graph is bipartite G' , we do not perturb an entry $\mathcal{A}[i, j]$ if the vertices i and j are in the same layer of G . Without loss of generality, we assume that the upper vertices have smaller IDs than the lower vertices. In this way, the adjacency matrix $\mathcal{A}$ can be divided into four blocks, where the ones on the diagonal are empty because the vertices on the same layer cannot be connected. In this work, we only apply RR to the upper right block to ensure the noisy graph is bipartite. For each vertex $u \in U(G)$, we flip the entries in $\mathcal{A}_u \cap L(G)$ that represent lower vertices. We denote the noisy graph w.r.t. a privacy budget ϵ as G'_ϵ .

Calibrating noise to global sensitivity. To ensure ϵ -edge LDP, any data transmitted from a vertex to the data curator requires the addition of noise. The Laplace Mechanism [36] is often used to draw noise proportional to the global sensitivity of the data. We formally define global sensitivity here.

Definition 4. Global sensitivity. Consider a bipartite graph G . Let $\mathcal{A}_u$ be the neighbor list of vertex u . The global sensitivity of a function $f : \mathcal{A}_u \rightarrow \mathbb{R}$ is:

$$\Delta f = \max_{u, u' \in V(G)} |f(\mathcal{A}_u) - f(\mathcal{A}'_u)|$$

Here, $\mathcal{A}'_u$ is a neighbor list that differs from $\mathcal{A}_u$ in at most one entry. Δf represents the global sensitivity of f .

Definition 5. The Laplace Mechanism. Given a privacy budget ϵ and any function f , the Laplace mechanism is defined as:

$$\tilde{f} = f + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

where $\tilde{f}$ is the noisy version of f and $\text{Lap}(\cdot)$ is the probability density function of the Laplace distribution.

Applying the Laplacian mechanism, we allow the vertices to send local graph statistics with calibrated noise to the data curator while satisfying ϵ -edge LDP.

Problem Statement. Given a bipartite graph G , a privacy budget ϵ , we aim to accurately and efficiently estimate the number of butterflies in G while satisfying ϵ -edge LDP.

Algorithm 1: Naive

Input: G : a bipartite graph; ϵ : a privacy budget;
Output: an estimate of $\mathbb{Z}_G$
// vertex side:
1 **foreach** $i \in U(G)$ **do**
2 **foreach** $j \in L(G)$ **do**
3 perturb $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^\epsilon} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$
4 send noisy edges to the data curator;
// curator side:
5 $G'_\epsilon \leftarrow$ the noisy graph constructed from $\mathcal{A}'[i, j]$;
6 **return** the butterfly count on G'_ϵ ;

A naive approach. Since the randomized responses preserve ϵ -edge LDP, a naive solution is to return the butterfly count of the noisy graph from the randomized responses, as detailed in Algorithm 1. Here, we use $\mathcal{A}'$ to indicate the adjacency matrix for the noisy graph G' . In Lines 1-4, each upper vertex perturbs its neighbor list and sends it to the data curator. Specifically, given a privacy budget $\epsilon > 0$, each entry $\mathcal{A}[i, j]$ with i and j being vertices on different layers is flipped with a probability $\frac{1}{1+e^\epsilon}$. In Lines 5-6, the data curator constructs a noisy graph G'_ϵ and returns the number of butterflies on G'_ϵ obtained by calling the state-of-the-art butterfly counting algorithm [6]. However, since most neighbor lists are sparse (with more “0” than “1”), flipping them with a fixed probability $\frac{1}{1+e^\epsilon}$ makes G'_ϵ much denser than G , which leads to severe over-counting.

In the remainder of this paper, we propose novel privacy-preserving algorithms under edge LDP to yield unbiased

estimates of butterfly counts. In Section III, we design a multiple-round algorithm and in Section IV, we present a download-free algorithm with improved data utility.

III. A MULTI-ROUND ALGORITHM

The Naive algorithm in Section II leads to poor data utility because the noisy graph is much denser than the original graph. Motivated by existing works on triangle counting under ϵ -edge LDP [29], [33], we adopt the multi-round interaction scheme to tackle over-counting butterflies due to the “dense noisy graph” issue. Specifically, we propose the MultiR algorithm to produce unbiased estimates of butterfly count by allowing the vertices and the data curator to interact for three rounds. In the first round, the algorithm estimates vertex degrees; in the second round, it constructs a noisy graph; and in the third round, the vertices download the noisy graph to compute their local motif counts, which are then aggregated to produce unbiased butterfly estimates. Note that each round consumes a privacy budget of ϵ_1 , ϵ_2 , and ϵ_3 , respectively, which adds up to ϵ . This ensures that the algorithm satisfies ϵ -edge LDP by the property of sequential composition.

A. An unbiased estimate for $\mathbb{Z}_G$

Under edge LDP, each vertex can only observe its neighbors locally, which prevents the data curator from extracting useful graph statistics from each vertex. Assuming ϵ_2 has been used to generate the noisy graph G'_{ϵ_2} in round 2, each vertex will download G'_{ϵ_2} that still preserves some structural information from the input graph. In this part, we leverage both local connections of vertices and the noisy graph structure by investigating the “local butterflies” and “local caterpillars” containing each vertex, which have two true edges in the input graph and the remaining edges in the noisy graph. These local motif counts are vital for the statistical inference of $\mathbb{Z}_G$.

local butterflies. We start with the local butterflies formed by $\langle u, v, w, x \rangle$, where the edges $(u, v), (u, w) \in E(G)$ and $(x, v), (x, w) \in E(G'_{\epsilon_2})$, comprising two noisy edges in G'_{ϵ_2} and two true edges in G connected to u . Let α_u denote the count of such butterflies containing u . Here, we investigate the expected value of the sum of α_u over all vertices, as given by

$$\mathbb{E}\left(\sum_{u \in V(G)} \alpha_u\right) = \sum_{\substack{v, w \in N(u, G) \\ u \neq x, v > w}} \mathcal{A}[u, v] \mathcal{A}[u, w] \mathbb{E}(\mathcal{A}'[x, v] \mathcal{A}'[x, w])$$

Here, $\mathcal{A}[i, j]$ denotes if vertices i and j are connected in G , and $\mathcal{A}'[i, j]$ denotes if they are connected in G'_{ϵ_2} . Obviously, the expectation of $\mathcal{A}'[i, j]$ depends on $\mathcal{A}[i, j]$, as given by

$$\mathbb{E}(\mathcal{A}'[i, j]) = \begin{cases} p, & \text{if } \mathcal{A}[i, j] = 0 \\ 1 - p & \text{if } \mathcal{A}[i, j] = 1 \end{cases}$$

Note that the flipping probability $p = 1/(1 + e^{\epsilon_2})$. Since the edges are perturbed independently, we can have

$\mathbb{E}(\mathcal{A}'[x, v]\mathcal{A}'[x, w]) = \mathbb{E}(\mathcal{A}'[x, v])\mathbb{E}(\mathcal{A}'[x, w])$. It readily follows that

$$\begin{aligned} \mathbb{E}\left(\sum_{u \in V(G)} \alpha_u\right) &= (1-p)^2 \sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |N(v) \cap N(w) \setminus \{u\}| \\ &+ p(1-p) \sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |N(v) \Delta N(w)| \\ &+ p^2 \sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |\{x \in V(G) : x \notin N(v) \cup N(w), x \neq u\}| \end{aligned} \quad (1)$$

where x and u resides on the same layer of G in the last term. $|N(v) \Delta N(w)|$ represents the symmetric difference between $N(v)$ and $N(w)$ (i.e., $N(v) \Delta N(w) = (N(v) \setminus N(w)) \cup (N(w) \setminus N(v))$). To simplify Equation 1, we derive the following lemma.

Lemma 1. *Given a bipartite graph G , we have*

$$\sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |N(v) \cap N(w) \setminus \{u\}| = 4 \mathbf{x}_G \quad (2)$$

$$\sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |N(v) \Delta N(w)| = 2(\mathbf{x}_G - 4 \mathbf{z}_G) \quad (3)$$

$$\begin{aligned} &\sum_{u \in V(G)} \sum_{\substack{v, w \in N(u) \\ v > w}} |\{x \notin N(v) \cup N(w), x \text{ and } u \text{ in same layer}\}| \\ &= \sum_{u \in U(G)} \binom{\deg(u)}{2} (n_1 - 1) + \sum_{v \in L(G)} \binom{\deg(v)}{2} (n_2 - 1) \\ &- 2 \mathbf{x}_G + 4 \mathbf{z}_G \end{aligned} \quad (4)$$

Here $\mathbf{x}_G$ and $\mathbf{z}_G$ denote the butterfly and caterpillar counts, respectively. A caterpillar is a 3-path in G .

Proof. In the following proof, we use (v, u, w) to denote a 2-path with u as the middle vertex.

To derive Equation (2): On the left-hand-side of Equation (2), both u and x are connected to v and w , which form a butterfly in G . The coefficient of 4 comes from repeatedly counting butterflies from their four vertices.

To derive Equation (3): The left-hand side of Equation (3) can be split into: $\sum_{(v, u, w)} |N(v) \setminus N(w)|$ and $\sum_{(v, u, w)} |N(w) \setminus N(v)|$, representing the caterpillars not contained in any butterflies. Since each butterfly can contain four caterpillars, these terms are reduced to $2(\mathbf{x}_G - 4 \mathbf{z}_G)$, where the coefficient of two comes from repeatedly counting each caterpillar twice.

To derive Equation (4): The left-hand side of Equation (4) can be reduced to

$$\begin{aligned} &\sum_{\substack{u \in U(G) \\ (v, u, w)}} |U(G) \setminus \{u\}| + \sum_{\substack{u \in L(G) \\ (v, u, w)}} |L(G) \setminus \{u\}| \\ &- \sum_{(v, u, w)} |N(v) \cup N(w) \setminus \{u\}| \end{aligned}$$

For the vertex u , the number of 2-paths centered at u is $\binom{\deg(u)}{2}$. Thus, the first and second terms are $(n_1 - 1) \sum_{u \in U(G)} \binom{\deg(u)}{2} + (n_2 - 1) \sum_{u \in L(G)} \binom{\deg(u)}{2}$. Based on Equation (3), the third term is $2 \mathbf{x}_G - 4 \mathbf{z}_G$. Putting these together gives Equation (4). $\square$

local caterpillars. We also investigate the local caterpillars formed by $\langle u, v, w, x \rangle$, where the edges $(u, v), (u, w) \in E(G)$ and either (x, v) or $(x, w) \in E(G'_{\epsilon_2})$. Let β_u denote the number of such caterpillars containing u . We compute the expected value of the sum of β_u over all vertices, as given by

$$\mathbb{E}\left(\sum_{u \in V(G)} \beta_u\right) = \sum_{u \neq x, v > w} \mathcal{A}[u, v]\mathcal{A}[u, w]\mathbb{E}(\mathcal{A}'[x, v] + \mathcal{A}'[x, w])$$

Following a similar logic for local butterflies and Lemma 1, we have the following equation for $\mathbb{E}(\sum_{u \in V(G)} \beta_u)$.

$$\begin{aligned} \mathbb{E}\left(\sum_{u \in V(G)} \beta_u\right) &= 8(1-p) \mathbf{x}_G + 2(\mathbf{x}_G - 4 \mathbf{z}_G) + \\ &2p\left(\sum_{u \in U(G)} \binom{\deg(u)}{2} (n_1 - 1) + \sum_{v \in L(G)} \binom{\deg(v)}{2} (n_2 - 1)\right) \\ &- 4p \mathbf{x}_G + 8p \mathbf{z}_G \end{aligned} \quad (5)$$

To compensate for the last term in $\mathbb{E}(\sum_{u \in V(G)} \alpha_u)$ and $\mathbb{E}(\sum_{u \in V(G)} \beta_u)$, for each vertex u , we also compute

$$\gamma_u = \begin{cases} \binom{\deg(u, G)}{2} (n_1 - 1), & \text{if } u \in U(G) \\ \binom{\deg(u, G)}{2} (n_2 - 1), & \text{if } u \in L(G) \end{cases} \quad (6)$$

Putting these together, we obtain an unbiased estimate for $\mathbf{x}_G$.

Theorem 2. *Given a bipartite graph G and a privacy budget ϵ_2 , let G'_{ϵ_2} be the noisy graph w.r.t. ϵ_2 . We have*

$$\mathbb{E}\left(\frac{\sum_u (\alpha_u - p\beta_u + p^2\gamma_u)}{4(1-2p)^2}\right) = \mathbf{x}_G$$

where $p = 1/(1 + e^{\epsilon_2})$.

Proof. The above formula naturally derives from the expected values of $\sum \alpha_u$, $\sum \beta_u$, and $\sum \gamma_u$, as elaborated in Equation (1), Equation (5), and Equation (6). $\square$

B. Global sensitivity analysis

According to Theorem 2, $w_u = \alpha_u - p\beta_u + p^2\gamma_u$ is computed for each vertex u and is sent to the data curator for aggregation. Note that we must add noise to w_u to satisfy ϵ -edge LDP. To do so, we analyze the global sensitivity of w_u and apply the Laplacian mechanism. Without loss of generality, we assume $u \in U(G)$. By Definition 4, the global sensitivity of w_u is $\Delta(w_u) = \max_{u, u' \in V(G)} |w_u - w_{u'}|$. Here, u and u' represent two vertices whose neighbor lists differ at one entry. Let β_u denote the number of local caterpillars

connected to vertex u , with u connected to two vertices in the caterpillar, given by $\tilde{\beta}_u = \beta_u - 2\alpha_u$. We have the following:

$$\begin{aligned}\Delta(w_u) &= \max_{u,u'} |(\alpha_u - \alpha'_u) - p(\beta_u - \beta'_u) + p^2(\gamma_u - \gamma'_u)| \\ &\leq \max_{u,u'} |(1-2p)|\alpha_u - \alpha'_u| + p|\tilde{\beta}_u - \tilde{\beta}'_u| + p^2|\gamma_u - \gamma'_u| \quad (7)\end{aligned}$$

We derive the following lemma to obtain a bound for $\Delta(w_u)$.

Lemma 2. *Given a bipartite graph G and a privacy budget ϵ_2 , let G'_{ϵ_2} be a noisy graph resulting from applying randomized responses to G . For a vertex $u \in U(G)$, we have the following inequalities for the global sensitivity of α_u , $\tilde{\beta}_u$, and γ_u .*

$$\max_{u,u'} |\alpha_u - \alpha'_u| \leq \deg(u, G) d_{\max}(L) \quad (8)$$

$$\max_{u,u'} |\tilde{\beta}_u - \tilde{\beta}'_u| \leq \deg(u, G) d_{\max}(L) + \sum_{v \in N(u, G)} \deg(v, G'_{\epsilon_2}) \quad (9)$$

$$\max_{u,u'} |\gamma_u - \gamma'_u| \leq \deg(u, G)(n_1 - 1) \quad (10)$$

Here $d_{\max}(L)$ represents the maximum vertex degree in $L(G)$.

Proof. To analyze the global sensitivities for $u \in U(G)$, we introduce a vertex $u' \in U(G)$ that connects to all neighbors of u and one additional vertex v^* (i.e., $N(u, G) \subseteq N(u', G)$ and $\deg(u', G) = \deg(u, G) + 1$). First, we prove Inequality (8). As shown in Fig. 2 (a), if each neighbor of u (in grey shade) is connected to each neighbor of v^* (in dashed line), u' participates in more butterflies than u and the difference is bounded by $|\alpha'_u - \alpha_u| = \deg(u, G) \times \deg(v^*, G'_{\epsilon_2}) \leq \deg(u, G) d_{\max}(L)$.

The left-hand-side of Inequality (9) represents the difference between the count of local caterpillars containing u and those containing u' . There are two types of such local caterpillars: (I) those containing a vertex in $N(v, G) \setminus N(v^*, G'_{\epsilon_2})$, and (II) those containing a vertex in $N(v^*, G'_{\epsilon_2}) \setminus N(v, G)$. For instance, as shown in Fig. 2 (b), $\langle v, u', v^*, x_5 \rangle$ forms a Type I caterpillar, while $\langle x_1, v, u', v^* \rangle$ is a Type II caterpillar. The difference in the number of Type I caterpillars is bounded by $\deg(u, G) d_{\max}(L)$ because one end-point v is chosen from $N(v, G)$ (in grey shade) and the other end-point is chosen from $N(v, G) \setminus N(v^*, G'_{\epsilon_2})$, which has at most $d_{\max}(L)$ candidates. As for Type II caterpillars, the difference is bounded by $\sum_{v \in N(u, G)} |N(v, G'_{\epsilon_2}) \setminus N(v^*, G'_{\epsilon_2})| \leq \sum_{v \in N(u, G)} \deg(v, G'_{\epsilon_2})$. Thus, Inequality (9) holds.

For Inequality (10), since u' is connected to $N(u) \cup \{v^*\}$, we have $\deg(u', G) = \deg(u, G) + 1$, which leads to:

$$\begin{aligned}|\gamma_u - \gamma'_u| &= (n_1 - 1) \left| \binom{\deg(u, G)}{2} - \binom{\deg(u, G) + 1}{2} \right| \\ &= \deg(u, G)(n_1 - 1)\end{aligned}$$

□

Based on Lemma 2, the upper bound for $\Delta(w_u)$ from

Inequality 7 becomes ($u \in U(G)$):

$$\begin{aligned}\Delta(w_u) &\leq (1-p)\deg(u, G)d_{\max}(L) + p\left(\sum_{v \in N(u, G)} \deg(v, G'_{\epsilon_2})\right) \\ &\quad + p^2\deg(u, G) \cdot (n_1 - 1)\end{aligned}$$

To avoid leaking $\deg(u, G)$, we estimate it using the Laplacian mechanism with a privacy budget of ϵ_1 and the global sensitivity of 2, i.e., $\tilde{d}(u) = \deg(u, G) + \text{Lap}(2/\epsilon_1)$. Here the global sensitivity of vertex degrees is two because one edge can increase the degree of at most two vertices, each by one [22], [38], [34]. However, using $\tilde{d}(u)$ in the above inequality about $\Delta(w_u)$ causes two issues: (i) $\tilde{d}(u)$ may not be positive, and (ii) $\tilde{d}(u)$ may be less than $\deg(u, G)$, which defeats the purpose of upper-bounding $\Delta(w_u)$. To address these, we adopt the widely-used “graph projection” technique under differential privacy [25], [39], [40], [33]. By adding a small constant κ to $\tilde{d}(u)$, we ensure that $\deg(u, G) \leq \tilde{d}(u) + \kappa$ with high probability. In rare cases where $\deg(u, G) > \tilde{d}(u) + \kappa$, we only use $\tilde{d}(u) + \kappa$ neighbors from $N(u)$ for local counting (effectively capping the degree of u at $\tilde{d}(u) + \kappa$). In this way, we can safely substitute $\deg(u, G)$ with $\tilde{d}(u) + \kappa$ in the global sensitivity analysis above.

C. The MultiR algorithm

In this part, we design a multi-round algorithm `MultiR` to produce unbiased estimates for $\mathbf{x}_G$, as outlined in Algorithm 2. In the first round, each vertex adds a small constant κ and some Laplacian noise calibrated to ϵ_1 to its degree and sends it to the data curator (Lines 2-3). This step aims to obtain fine-grained upper bounds for vertex degrees, which is used in the estimation of $\Delta(w_u)$ in round 3. In the second round, Algorithm 2 applies randomized responses w.r.t. ϵ_2 and constructs a noisy graph G'_{ϵ_2} (Lines 4-8). In the third round, each vertex u receives the noisy graph and computes the counts of local butterflies and caterpillars, i.e., α_u and β_u (Lines 9-25). To improve efficiency, we extend the wedge-based method in [6] to handle local butterflies and caterpillars in batches. Specifically, given a vertex u , for each vertex w , we count the wedges (i.e., 2-paths) $\langle u, v, w \rangle$ such that $(u, v) \in G$ and $(v, w) \in G'_{\epsilon_2}$ and store it in $\text{count_wedge}[w]$ (Lines 12-15). In this way, α_u and β_u are computed as $\sum_w \binom{\text{count_wedge}[w]}{2}$ and $\sum_w \text{count_wedge}[w] \deg(u, G)$, respectively (Lines 16-19). The compensation term γ_u is computed in Lines 20-23. Using these counts, we can calculate w_u and add Laplacian noise to w_u calibrated to $\Delta(w_u)$ (Lines 24, 25). Note that the Laplacian noise has an expected value of zero so Algorithm 2 still produces an unbiased estimate of $\mathbf{x}_G$ via Theorem 2.

Compared to the naive solution, the `MultiR` algorithm addresses the “dense noisy graph” issue by incorporating the edge flipping probability ($p = 1/(1+e^{\epsilon_2})$) to estimate butterfly counts. This ensures unbiased results regardless of how dense the noise graph becomes. In addition, based on the sequential composition property, `MultiR` still satisfies ϵ -edge LDP.

IV. A DOWNLOAD-FREE BUTTERFLY ESTIMATION ALGORITHM

Algorithm 2: The MultiR algorithm

Input: G : a bipartite graph; ϵ : a privacy budget; κ : graph projection constant

Output: a private estimate of $\mathbb{X}_G$

- 1 split privacy budget ϵ into ϵ_1 , ϵ_2 and ϵ_3 ;
- // **round 1:**
- 2 **foreach** $u \in V(G)$ **do**
- 3 $\tilde{d}(u) = \deg(u, G) + \text{Lap}(\frac{2}{\epsilon_1}) + \kappa$;
- // **round 2:**
- 4 $p \leftarrow \frac{1}{1+e^{\epsilon_2}}$;
- 5 **foreach** $i \in U(G)$ **do**
- 6 **foreach** $j \in L(G)$ **do**
- 7 perturb $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^{\epsilon_2}}; \\ \mathcal{A}[i, j], & \text{w.p. } \frac{e^{\epsilon_2}}{1+e^{\epsilon_2}} \end{cases}$;
- 8 $G'_{\epsilon_2} \leftarrow$ the noisy graph constructed from $\mathcal{A}'[i, j]$;
- // **round 3:**
- 9 $sum \leftarrow 0$;
- 10 **for** $u \in V(G)$ **do**
- 11 $\alpha_u \leftarrow 0, \beta_u \leftarrow 0, \gamma_u \leftarrow 0$;
- 12 initialize hashmap $count_wedge$ with zero;
- 13 **for** $v \in N(u, G)$ **do**
- 14 **for** $w \in N(v, G'_{\epsilon_2})$ and $w \neq u$ **do**
- 15 $count_wedge[w] \leftarrow count_wedge[w] + 1$;
- 16 **for** $w \in count_wedge$ **do**
- 17 **if** $count_wedge[w] > 1$ **then**
- 18 $\alpha_u \leftarrow \alpha_u + \binom{count_wedge[w]}{2}$;
- 19 $\beta_u \leftarrow \beta_u + count_wedge[w](\deg(u, G) - 1)$;
- 20 **if** $u \in U(G)$ **then**
- 21 $\gamma_u \leftarrow \binom{\deg(u)}{2} \times (n_1 - 1)$;
- 22 **else**
- 23 $\gamma_u \leftarrow \binom{\deg(u)}{2} \times (n_2 - 1)$;
- 24 $w_u \leftarrow \alpha_u - p\beta_u + p^2\gamma_u$;
- 25 $sum \leftarrow sum + w_u + \text{Lap}(\frac{\Delta(w_u)}{\epsilon_3})$;
- 26 **return** $\frac{sum}{4(1-2p)^2}$;

Although the MultiR algorithm gives unbiased estimates for butterfly counts in Section III, it injects substantial noise due to the high global sensitivity of local counts, which undermines data utility. In triangle counting with edge LDP, a non-interactive algorithm is also proposed, which uses motif counts on the noisy graph to statistically infer the triangle count [29], [33]. However, such an approach is outperformed by its multi-round competitor because the latter tightly bound the global sensitivity for minimal information loss during data transmission. In the meantime, on bipartite graphs, we also observe that this non-interactive approach can escape the “curse of high global sensitivity” in butterfly counting because it only relies on randomized responses to meet privacy requirements. Thus, in this section, we resort to a non-interactive approach and propose a Download-free Butterfly Estimation (DBE) algorithm that closely models the motif transformation probabilities during randomized responses and provides an unbiased estimate of the butterfly count that is more practically applicable.

A. Motif transformation during Randomized responses

When applying randomized responses to a bipartite graph G , the resulting noisy graph G' becomes considerably denser,

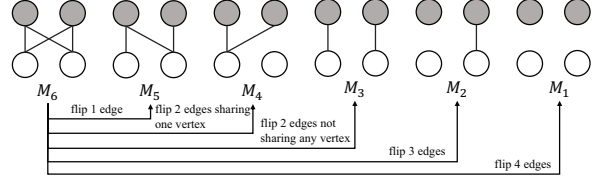


Fig. 3: All motifs with two upper and lower vertices

and the butterfly count significantly increases. To recover the butterfly count on G , we must investigate how the motifs with two upper and lower vertices transform into each other with different probabilities. Here, we formally define the transformation probability as such.

Definition 6. Transformation probability. Given a privacy budget ϵ and a bipartite graph G , let G'_ϵ be a noisy graph from applying randomized response to G . Let $\mathcal{M}$ be a motif in G and $\mathcal{N}$ be a motif in G'_ϵ . The probability that the motif in G'_ϵ induced by the vertices in $\mathcal{M}$ is $\mathcal{N}$ is called the transformation probability from $\mathcal{M}$ to $\mathcal{N}$, i.e.,

$$TP(\mathcal{M}, \mathcal{N}) = P(G'[V] \cong \mathcal{M} \mid G[V] \cong \mathcal{N})$$

Here $\mathcal{V}$ denotes a vertex set in $V(G)$. The symbol $\cong$ denotes graph isomorphism [41] between two graphs, indicating a bijective edge-invariant vertex mapping between them.

As shown in Fig. 3, a butterfly can transform into any of the six motifs, denoted by $(\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_6)$. Here, $\mathcal{M}_6$ is the butterfly itself. The transformation probability from a butterfly to itself is $(1-p)^4$ as it requires each of the 4 edges to stay un-flipped during randomized responses ($p = 1/(1+e^\epsilon)$). Based on Definition 6, we derive the following lemma, which establishes a connection between the expected values of motif counts on G' and those on G .

Lemma 3. Let $n_{\mathcal{M}_i}$ denote the number of instances of $\mathcal{M}_i$ on G and $n'_{\mathcal{M}_i}$ denote their counts on G' . We have:

$$\mathbb{E}(n'_{\mathcal{M}_1}, n'_{\mathcal{M}_2}, \dots, n'_{\mathcal{M}_6}) = (n_{\mathcal{M}_1}, n_{\mathcal{M}_2}, \dots, n_{\mathcal{M}_6}) \times \mathcal{T}$$

$\mathcal{T}$ is a 6×6 matrix, where $\mathcal{T}[i, j] = TP(\mathcal{M}_i, \mathcal{M}_j)$.

Proof. We start by investigating the expected value of $n'_{\mathcal{M}_i}$:

$$\mathbb{E}(n'_{\mathcal{M}_i}) = \sum_{\mathcal{V} \in V(G)} P(G'[\mathcal{V}] \cong \mathcal{M}_i)$$

To compute $P(G'[\mathcal{V}] \cong \mathcal{M}_i)$, we consider all motifs that can be transformed into $\mathcal{M}_i$ and apply the law of total probability:

$$P(G'[\mathcal{V}] \cong \mathcal{M}_i) = \sum_{1 \leq j \leq 6} TP(\mathcal{M}_j, \mathcal{M}_i) P(G[\mathcal{V}] \cong \mathcal{M}_j)$$

We substitute $P(G'[\mathcal{V}] \cong \mathcal{M}_i)$ into $\mathbb{E}(n'_{\mathcal{M}_i})$ and obtain $\mathbb{E}(n'_{\mathcal{M}_i}) = \sum_{1 \leq j \leq 6} TP(\mathcal{M}_j, \mathcal{M}_i) n_{\mathcal{M}_j}$.

Note that this equation holds because $n_{\mathcal{M}_j} = \sum_{G[\mathcal{V}] \cong \mathcal{M}_j} 1$. In this way, we obtain a linear system with $n_{\mathcal{M}_i}$ as variables $\mathbb{E}(n'_{\mathcal{M}_1}, \dots, n'_{\mathcal{M}_6}) = (n_{\mathcal{M}_1}, \dots, n_{\mathcal{M}_6}) \times \mathcal{T}$. $\square$

Computing the transformation probabilities It is difficult

to compute each element in $\mathcal{T}$ manually. To simplify this, we design a TP-Computation algorithm that computes the transformation probabilities for every pair of motifs $\mathcal{M}_i$ and $\mathcal{M}_j$ systematically, as outlined in Algorithm 3. First, it

Algorithm 3: TP-Computation

Input: G : a bipartite graph; ϵ : a privacy budget
Output: The transformation probability matrix $\mathcal{T}$

```

1  $\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_6 \leftarrow$  distinct motif types with 2 upper
   vertices and 2 lower vertices;
2  $\mathcal{T} \leftarrow$  a 6 by 6 matrix with zeros;
3  $p \leftarrow \frac{1}{1+\epsilon}$ ;
4 foreach  $\mathcal{M}_i \in \{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_6\}$  do
5   for  $x \in [0, 4]$  do
6     foreach  $\mathcal{M}_j$  with  $x$  entries flipped do
7        $\mathcal{T}[i][j] = \mathcal{T}[i][j] + p^x(1-p)^{4-x}$ ;
8 return  $\mathcal{T}$ 
```

initializes an all-zero 6×6 matrix ($\mathcal{T}$) and computes the flipping probability p based on the privacy budget ϵ (Lines 1-3). Note that each $\mathcal{M}_i$ corresponds to four entries in the adjacency matrix, each of which can be flipped by randomized responses. Thus, for $\mathcal{M}_i$, the algorithm considers each possible variation $\mathcal{M}_j$ where x entries are flipped while $4-x$ entries remain unchanged ($0 \leq x \leq 4$) and increments the $\mathcal{T}[i, j]$ by $p^x(1-p)^{4-x}$ (Lines 4-7). After all possibilities are explored, $\mathcal{T}$ stores the accurate transformation probabilities as follows.

$$\mathcal{T} = \frac{1}{(1+\mu)^4} \times \begin{pmatrix} \mu^4 & 4\mu^3 & 2\mu^2 & 4\mu^2 & 4\mu & 1 \\ \mu^3 & \mu^4 + 3\mu^2 & \mu^3 + \mu & 2\mu^3 + 2\mu & 3\mu^2 + 1 & \mu \\ \mu^2 & 2\mu^3 + 2\mu & \mu^4 + 1 & 4\mu^2 & 2\mu^3 + 2\mu & \mu^2 \\ \mu^2 & 2\mu^3 + 2\mu & 2\mu^2 & \mu^4 + 2\mu^2 + 1 & 2\mu^3 + 2\mu & \mu^2 \\ \mu & 3\mu^2 + 1 & \mu^3 + \mu & 2\mu^3 + 2\mu & \mu^4 + 3\mu^2 & \mu^3 \\ 1 & 4\mu & 2\mu^2 & 4\mu^2 & 4\mu^3 & \mu^4 \end{pmatrix}$$

and $\mu = e^\epsilon$.

Example 1. Here we demonstrate the computation of $\mathcal{T}[4, 4]$, the transformation probability of $\mathcal{M}_4$ to itself. Let x denote the number of flipped entries in $\mathcal{M}_4$. Three possibilities arise: (a) $x = 0$. No entry is flipped, which happens with a probability $(1 - \frac{1}{1+\mu})^4$. (b) $x = 2$, and the two flipped entries do not share any vertex. This happens with a probability of $2 \times (\frac{1}{1+\mu})^2 (\frac{\mu}{1+\mu})^2$. (c) $x = 4$. All entries are flipped, which happens with a probability $(1 - \frac{1}{1+\mu})^4$. Summing these probabilities yields $\frac{(\mu^2+1)^2}{(\mu+1)^4}$.

Since $n_{\mathcal{M}_6} = \mathbf{z}_G$, we solve for $n_{\mathcal{M}_6}$ and obtain the unbiased estimate for $\mathbf{z}_G$ in the theorem below.

Theorem 3. Given a bipartite graph G , a privacy budget ϵ , let G' be the noisy graph after applying randomized response to G w.r.t ϵ . We have $\mathbb{E}(f_{\mathbf{z}}(G')) = \mathbf{z}_G$ where

$$f_{\mathbf{z}}(G') = \frac{1}{(\mu-1)^4} (\mu^4 n'_{\mathcal{M}_6} - \mu^3 n'_{\mathcal{M}_5} + \mu^2 (n'_{\mathcal{M}_4} + n'_{\mathcal{M}_3}) - \mu n'_{\mathcal{M}_2} + n'_{\mathcal{M}_1}) \quad (11)$$

is the unbiased estimate for $\mathbf{z}_G$ ($\mu = e^\epsilon$).

Proof. By Lemma 3, the unbiased estimators for $n_{\mathcal{M}_i}$ are expressed as $(\hat{n}_{\mathcal{M}_1}, \hat{n}_{\mathcal{M}_2}, \dots, \hat{n}_{n_{\mathcal{M}_6}}) = (n'_{\mathcal{M}_1}, n'_{\mathcal{M}_2}, \dots, n'_{\mathcal{M}_6}) \times \mathcal{T}^{-1}$. Among them, the sixth term $\hat{n}_{\mathcal{M}_6}$ provides the estimate for $\mathbf{z}_G$. Equation 11 is obtained by multiplying the noisy motif counts $n_{\mathcal{M}_i}$ ($1 \leq i \leq 6$) by the sixth column of $\mathcal{T}^{-1}$. $\square$

B. Obtaining the noisy motif counts

In the previous part, we examine the transformation probabilities among the motifs with two upper and lower vertices and establish an unbiased estimator for the butterfly count. However, how to obtain the motif counts ($n'_{\mathcal{M}_i}$) on the noisy graph G' remains to be addressed. In this part, we derive specific formulas to efficiently obtain these motif counts.

Lemma 4. Given a bipartite graph G , let m be the number of edges. n_1 and n_2 represent the number of upper and lower vertices, respectively. Let d_u and d_v represent the degrees of u and v . The following equations hold:

$$n_{\mathcal{M}_6} = \mathbf{z}_G \quad (12)$$

$$n_{\mathcal{M}_5} = \sum_{(u,v) \in E(G)} (d_u - 1)(d_v - 1) - 4 \times \mathbf{z}_G \quad (13)$$

$$n_{\mathcal{M}_4} = (n_1 - 1) \sum_{v \in U(G)} \binom{d_v}{2} + (n_2 - 1) \sum_{v \in L(G)} \binom{d_v}{2} - 4 \times \mathbf{z}_G - 2n_{\mathcal{M}_5} \quad (14)$$

$$n_{\mathcal{M}_3} = \frac{1}{2} \sum_{(u,v) \in E(G)} (m - d_u - d_v + 1) - 2n_{\mathcal{M}_6} - n_{\mathcal{M}_5} \quad (15)$$

$$n_{\mathcal{M}_2} = m(n_1 - 1)(n_2 - 1) - 4 \times \mathbf{z}_G - 3n_{\mathcal{M}_5} - 2n_{\mathcal{M}_3} - 2n_{\mathcal{M}_4} \quad (16)$$

$$n_{\mathcal{M}_1} = \binom{n_1}{2} \times \binom{n_2}{2} - \sum_{2 \leq i \leq 6} n_{\mathcal{M}_i} \quad (17)$$

Proof. Equation 12 holds since $\mathcal{M}_6$ is the butterfly itself. Equation 13 holds because $\mathcal{M}_5$ represents the 3-paths not contained in any butterflies. The total number of 3-paths is computed as $\sum_{(u,v) \in E(G)} (d_u - 1)(d_v - 1)$, where $(d_u - 1)(d_v - 1)$ is the number of 3-paths with (u, v) as the middle edge. Since each butterfly contains $\binom{4}{3} = 4$ number of 3-paths, the number of 3-paths contained in butterflies is $4 \times \mathbf{z}_G$. Thus, the number of $\mathcal{M}_5$ is $\sum_{(u,v) \in E(G)} (d_u - 1)(d_v - 1) - 4 \times \mathbf{z}_G$.

Note that $\mathcal{M}_4$ consists of a 2-path (v_1, u, v_2) and an isolated vertex x . Let u be the middle vertex of the 2-path. x must be from the same layer as u but it cannot be connected to v_1 or v_2 . This leads to the following expression:

$$n_{\mathcal{M}_4} = \sum_{u \in U(G)} |U(G) \setminus \{u\}| + \sum_{u \in L(G)} |L(G) \setminus \{u\}| - \sum_{(v_1, u, v_2)} |N(v_1) \setminus N(v_2) \setminus \{u\}|.$$

For the vertex u , the number of 2-paths with u being the middle vertex is $\binom{d_u}{2}$. Thus, the first two terms become $(n_1 - 1) \sum_{u \in U(G)} \binom{d_u}{2} + (n_2 - 1) \sum_{u \in L(G)} \binom{d_u}{2}$. Based on Lemma 1 and Equation 13, the third term can be reduced to $4 \times \mathbf{z}_G + 2n_{\mathcal{M}_5}$. Thus, Equation 14 holds.

Note that $\mathcal{M}_3$ consists of two components, each formed by two vertices connected by a single edge. Let u, v be two vertices in $\mathcal{M}_3$ where $(u, v) \in E(G)$. The additional two vertices x and w must satisfy: (1) $(x, w) \in E(G)$ (2) $x \neq u$ and $w \neq v$ and (3) $x \in U(G) \setminus N(v)$ and $w \in L(G) \setminus N(u)$. This leads to the following breakdown for $n_{\mathcal{M}_3}$.

$$\begin{aligned} 2 \times n_{\mathcal{M}_3} &= \sum_{(u,v) \in E(G)} |\{(x, w) \in E(G) : x \neq u, w \neq v, \\ &\quad x \in U(G) \setminus N(v), w \in L(G) \setminus N(u)\}| \\ &= \sum_{(u,v) \in E(G)} (m - d_u - d_v + 1 - 4 \mathbf{x}_G - 2n_{\mathcal{M}_5}) \end{aligned}$$

The coefficient “2” preceding $n_{\mathcal{M}_3}$ accounts for the double counting of each $\mathcal{M}_3$ from its two edges.

$\mathcal{M}_2$ consists of two vertices connected via one edge and two isolated vertices. Let u and v be two connected vertices in $\mathcal{M}_2$ (i.e., $(u, v) \in E(G)$). The other two vertices x and w should satisfy: (1) $x \in U(G) \setminus \{u\}$ and $x \notin N(v)$ and (2) $w \in L(G) \setminus \{v\}$ and $w \notin N(u) \cup N(x)$. This leads to the expression $n_{\mathcal{M}_2} = \sum_{(u,v)} \sum_x (|L(G) \setminus N(u) \setminus \{v\}| - |N(x) \setminus N(u) \setminus \{v\}|)$.

Note that the second term is precisely $2n_{\mathcal{M}_3}$. The first term can be further reduced to:

$$\begin{aligned} \sum_{(u,v)} (n_1 - d_v)(n_2 - d_v) &= m(n_1 - 1)(n_2 - 1) - \\ &\quad 2(n_1 - 1) \sum_{u \in U(G)} \binom{d_u}{2} - 2(n_2 - 1) \sum_{v \in L(G)} \binom{d_v}{2} \\ &\quad + \sum_{(u,v)} (d_u - 1)(d_v - 1) \end{aligned}$$

Thus, Equation 16 holds based on Equations 14 and 13.

For $\mathcal{M}_1$, we can exploit the fact that the number of the motifs $\mathcal{M}_i$ ($1 \leq i \leq 6$) sum up to $\binom{n_1}{2} \times \binom{n_2}{2}$. Hence, $n_{\mathcal{M}_1}$ is computed as $\binom{n_1}{2} \times \binom{n_2}{2} - \sum_{2 \leq i \leq 6} n_{\mathcal{M}_i}$. $\square$

C. The DBE algorithm

Building upon prior discussions, we present the DBE algorithm as detailed in Algorithm 5, which involves computation on both the vertex side and the data curator side. On the vertex side, each vertex applies randomized responses to its adjacency list and sends the noisy edges to the data curator (Lines 1-4). On the data curator side, it constructs a noisy graph G'_ϵ from these noisy edges and obtains an unbiased estimate for butterfly count using motif counts on G'_ϵ . The data curator performs this task by computing the inverse of the transformation probability matrix to obtain the formula for $\mathbf{x}_G$ (i.e., $\mathcal{T}^{-1}$ via Algorithm 3, Line 6). Note that this is a one-off task in practice, as the derived formula is used repeatedly. Subsequently, the data curator computes $n'_{\mathcal{M}_i}$ ($1 \leq i \leq 6$) by invoking Algorithm 4 on G'_ϵ (Line 7). In the end, Algorithm 5 computes an unbiased estimate for $\mathbf{x}_G$ via Theorem 3.

Theorem 4. *The DBE algorithm satisfies ϵ -edge LDP.*

Proof. The theorem follows because the randomized responses provide ϵ -edge LDP, which is immune to post-processing. $\square$

Algorithm 4: compute $n_{\mathcal{M}_i}$

Input: G : a bipartite graph; $\mathbf{x}_G$: the butterfly count in G
Output: $n_{\mathcal{M}_i}$ ($1 \leq i \leq 6$)

```

1  $n_{\mathcal{M}_6} \leftarrow$  the butterfly count on  $G$ ;
2  $n_{\mathcal{M}_5} \leftarrow 0, n_{\mathcal{M}_4} \leftarrow 0, n_{\mathcal{M}_3} \leftarrow 0$ ;
3  $n_1 \leftarrow |U(G)|, n_2 \leftarrow |L(G)|$ ;
  // Compute  $n_{\mathcal{M}_5}$ 
4 foreach  $(u, v) \in E(G)$  do
5    $n_{\mathcal{M}_5} \leftarrow n_{\mathcal{M}_5} + (deg(u, G) - 1)(deg(v, G) - 1)$ ;
6  $n_{\mathcal{M}_5} \leftarrow n_{\mathcal{M}_5} - 4 \mathbf{x}_G$ ;
  // Compute  $n_{\mathcal{M}_4}$ 
7 for  $u \in V(G)$  do
8   if  $u \in U(G)$  then
9      $n_{\mathcal{M}_4} \leftarrow n_{\mathcal{M}_4} + (n_1 - 1) \binom{deg(u, G)}{2}$ ;
10  else
11     $n_{\mathcal{M}_4} \leftarrow n_{\mathcal{M}_4} + (n_2 - 1) \binom{deg(u, G)}{2}$ ;
12  $n_{\mathcal{M}_4} \leftarrow n_{\mathcal{M}_4} - 4 \mathbf{x}_G - 2n_{\mathcal{M}_5}$ ;
  // Compute  $n_{\mathcal{M}_3}$ 
13 foreach  $(u, v) \in E(G)$  do
14    $n_{\mathcal{M}_3} \leftarrow n_{\mathcal{M}_3} + |E(G)| - deg(u, G) - deg(v, G) + 1$ ;
15  $n_{\mathcal{M}_3} \leftarrow n_{\mathcal{M}_3} / 2 - 2 \mathbf{x}_G - n_{\mathcal{M}_5}$ ;
  // Compute  $n_{\mathcal{M}_2}$ 
16  $n_{\mathcal{M}_2} \leftarrow$ 
    $(n_1 - 1)(n_2 - 1)|E(G)| - 4 \mathbf{x}_G - 3n_{\mathcal{M}_5} - 2(n_{\mathcal{M}_4} + n_{\mathcal{M}_3})$ ;
  // Compute  $n_{\mathcal{M}_1}$ 
17  $n_{\mathcal{M}_1} \leftarrow \binom{n_1}{2} \binom{n_2}{2} - \mathbf{x}_G - n_{\mathcal{M}_5} - n_{\mathcal{M}_4} - n_{\mathcal{M}_3} - n_{\mathcal{M}_2}$ ;
18 return  $n_{\mathcal{M}_i}$   $1 \leq i \leq 6$ 
```

Algorithm 5: DBE-BTF

Input: G : a bipartite graph; ϵ : a privacy budget
Output: a private estimate of $\mathbf{x}_G$

// **vertex side:**

```

1 foreach  $i \in U(G)$  do
2   foreach  $j \in L(G)$  do
3      $\text{perturb } \mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+\epsilon} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{\epsilon}{1+\epsilon} \end{cases}$ 
4 send noisy edges to the data curator;
  // data curator side:
5  $G'_\epsilon \leftarrow$  the noisy graph constructed from  $\mathcal{A}'[i, j]$ ;
6 compute  $\mathcal{T}^{-1}$  via Algorithm 3;
7 compute  $n'_{\mathcal{M}_i}$  ( $1 \leq i \leq 6$ ) in  $G'_\epsilon$  via Algorithm 4;
8 return  $(\mu^4 \mathbf{x}_G - \mu^3 n'_{\mathcal{M}_5} + \mu^2 ((n'_{\mathcal{M}_4} + n'_{\mathcal{M}_3})) - \mu n'_{\mathcal{M}_2} + n'_{\mathcal{M}_1}) / (\mu - 1)^4$ 
```

Compared to Naive, DBE addresses the “dense noisy graph” issue by providing unbiased estimates for butterfly counts regardless of the noisy graph’s density. Compared to the MultiR algorithm, DBE yields more accurate butterfly count estimates by avoiding the need to inject excessive noise

TABLE II: Complexities of the butterfly counting algorithms

Algorithm	Time cost	Communication cost
Naive	$O(\frac{n_1 n_2}{1+\epsilon} + E(G'_\epsilon) ^{3/2})$	$O(E(G'))$
MultiR	$O(n_1 + n_2 + \frac{n_1 n_2}{1+\epsilon} + E(G'_\epsilon) ^{3/2})$	$O((n_1 + n_2) E(G'_\epsilon))$
DBE	$O(\frac{n_1 n_2}{1+\epsilon} + E(G'_\epsilon) ^{3/2})$	$O(E(G'))$

* ϵ is the privacy budget. G'_ϵ denotes a noisy graph w.r.t. ϵ . n_1 and n_2 denote the number of upper and lower vertices.

caused by the high global sensitivity of local motif counts.

Complexity analysis. Here, we summarize the total time and communication cost of Naive, MultiR, and DBE in Table II. For Naive and DBE, the time cost incurred by randomized response is $O(n_1 n_2 \times \frac{1}{1+\epsilon})$ since the flipping probability is $\frac{1}{1+\epsilon}$. On the data curator side, the dominating part of the time cost in Naive and DBE is incurred by butterfly counting on the noisy graph G'_ϵ , which takes $O(|E(G'_\epsilon)|^{3/2})$ time [6], [42]. The communication costs of Naive and DBE are both $O(|E(G'_\epsilon)|)$ because they need to send the noisy edges. As for MultiR, it requires $O(n_1 + n_2)$ time for vertex degree estimation in round 1 and $O(n_1 n_2 \times \frac{1}{1+\epsilon^2})$ time for noisy graph construction in round 2. The dominating part of the communication cost is incurred by downloading the noisy graph to each vertex, which takes $O((n_1 + n_2)|E(G'_\epsilon)|)$.

V. EXPERIMENTAL EVALUATION

In this section, we evaluate the proposed butterfly counting algorithms under ϵ -edge LDP through experiments.

A. Experimental Settings

TABLE III: Summary of Datasets

Dataset	Upper	Lower	$ E $	$ U $	$ L $
Unicode (UN)	Country	Language	1.26K	255	615
Luri (LR)	User	Article	18.9K	209	8.74K
FilmTrust (LS)	User	Film	35.5K	1.5K	2.07K
Romansh (RM)	User	Article	57.9K	1.22K	8.09K
Condmatt (AC)	Author	Paper	58.6K	16.7K	22.0K
Kashubian (CS)	User	Article	62.1K	1.33K	8.28K
DailyKos (BK)	Document	Word	353K	3.43K	6.90K
Bishnupriya (BP)	User	Article	399.7K	1.32K	57.9K
NIPS (NS)	Document	Word	746.3K	1.50K	12.4K
LastFm (LB)	User	Band	898K	38.7M	16.7M
Discogs (DL)	Label	Style	1.06M	243.8K	0.38K
DiggVotes (DV)	User	Story	3.01M	139.4K	3.55K
MovieLens (MV)	User	Movie	10.0M	69.9K	10.7K
Netflix (NX)	User	Movie	100.5M	480.2K	17.8K

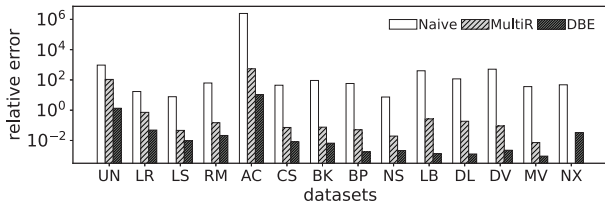


Fig. 4: Performance on different datasets ($\epsilon = 2$)

Datasets. We use 14 real datasets from KONECT (<http://konect.cc/>) in our experiments. Table III shows the statistics of the datasets. The columns “Upper” and “Lower” display the entities that upper vertices and lower vertices represent in each dataset. $|U|$ and $|L|$ are the numbers of vertices in the upper and lower layers, respectively. $|E|$ is the number of edges in the graph.

Algorithms. We evaluate the following algorithms:

- Naive: the algorithm that returns the butterfly count on the noisy graph;

TABLE IV: The time cost (s) of butterfly counting algorithms

Dataset	ϵ	Naive			MultiR				DBE		
		RR	curator side	total	RR	degree estimate	local counting	total	RR	curator side	total
CS	1	0.3139	15.62	15.94	0.4253	0.0079	3.92	4.35	0.2956	15.49	15.78
	1.5	0.3369	8.21	8.55	0.3963	0.0079	3.59	3.99	0.3225	7.74	8.06
	2	0.3052	3.77	4.07	0.2980	0.0063	3.16	3.46	0.2338	3.36	3.60
	2.5	0.2831	1.83	2.12	0.3436	0.0079	3.37	3.72	0.2717	1.81	2.08
BK	1	0.2670	0.9580	1.22	0.3887	0.0079	3.33	3.73	0.2066	0.74	0.95
	1.5	0.9680	61.96	62.93	1.13	0.0070	8.76	9.89	1.04	62.14	63.18
	1.5	1.01	31.74	32.75	1.07	0.0072	8.25	9.33	0.8710	31.18	32.05
	2	0.8250	15.51	16.34	1.00	0.0070	7.99	9.00	0.8523	15.52	16.37
BP	2.5	0.7976	8.13	8.93	0.9021	0.0068	7.13	8.04	0.7631	8.18	8.95
	3	0.7605	4.91	5.67	1.02	0.0085	6.67	7.70	0.8307	4.91	5.74
NS	1	5.96	429.44	435.40	7.31	0.0408	327.28	334.63	5.60	428.83	434.43
	1.5	5.90	214.24	220.14	6.21	0.0413	281.05	287.30	5.36	211.07	216.43
	2	5.83	99.08	104.91	5.64	0.0389	285.82	291.50	5.15	98.77	103.91
	2.5	5.37	45.71	51.07	5.54	0.0389	289.05	294.63	5.04	45.43	50.48
LB	3	5.31	22.84	28.15	6.26	0.0401	373.77	380.07	4.92	22.79	27.71
	1	2.35	69.71	72.06	2.35	0.0091	22.56	24.92	2.24	39.89	42.13
	1.5	3.30	23.68	26.97	2.27	0.0091	22.30	24.58	2.16	21.83	23.99
	2	2.42	13.31	15.73	2.24	0.0092	21.49	23.74	2.10	12.47	14.58
DL	2.5	2.36	7.91	10.27	2.30	0.0094	20.40	22.71	2.07	7.33	9.40
	3	2.19	5.42	7.61	2.30	0.0096	19.43	21.74	2.05	5.46	7.50
DV	1	38.16	2609.75	2647.91	37.26	0.1173	2061.08	2098.45	36.09	2593.05	2629.14
	1.5	37.55	1262.41	1299.96	36.83	0.1149	2120.10	2157.05	34.74	1262.43	1297.18
	2	36.69	575.72	612.41	37.00	0.1149	2043.79	2080.90	34.33	574.24	608.57
	2.5	36.49	248.92	285.41	35.97	0.1166	1842.87	1878.96	34.38	247.75	282.13
MV	3	36.25	105.97	142.22	35.47	0.1149	1581.68	1617.27	33.79	105.37	139.16
	1	1.84	1960.54	1962.38	10.45	0.0939	1982.98	1993.52	1.99	1969.37	1971.36
	1.5	1.55	974.84	976.39	9.79	0.0939	1810.40	1820.29	1.57	983.26	984.83
	2	1.31	463.95	465.26	2.06	0.1604	6172.48	6174.70	1.34	461.60	462.93
NX	2.5	1.17	215.20	216.38	1.83	0.1605	3844.96	3846.95	1.20	219.09	220.29
	3	1.07	105.68	106.75	1.67	0.1602	3595.21	3597.04	1.10	105.55	106.65
UN	1	11.40	6773.35	6784.75	40.36	0.0530	2431.75	2472.17	11.37	6738.90	6750.27
	1.5	9.73	3308.50	3318.23	36.80	0.0529	2173.13	2209.98	9.70	3295.49	3305.19
	2	8.50	1506.99	1515.49	11.54	0.0941	2276.94	2288.57	8.42	1507.80	1516.22
	2.5	7.61	665.06	672.67	10.38	0.0938	2009.51	2019.99	7.58	666.48	674.06
LR	3	7.02	293.20	300.22	9.73	0.0942	1834.03	1843.86	6.99	294.21	301.20
	1	38.68	9310.36	9349.05	41.96	0.0530	3201.92	3243.93	37.57	9235.54	9273.11
	1.5	35.21	4611.47	4646.68	40.22	0.0530	2322.19	2362.46	34.03	4610.03	4644.06
	2	33.25	2191.73	2224.98	38.82	0.0530	2332.76	2371.63	31.81	2188.28	2220.09
RM	2.5	31.30	1047.66	1078.96	36.82	0.0531	2224.34	2261.21	30.06	1049.90	1079.95
	3	30.33	558.24	588.56	35.26	0.0530	2568.55	2603.86	29.14	553.32	582.47

* RR stands for “randomized response”.

- MultiR: the proposed multi-round algorithm that allows the vertices to download the noisy graph and count local motifs on it;
- DBE: the proposed download-free algorithm that provides unbiased estimates of butterfly counts based on motif counts on the noisy graph;

All algorithms are implemented in C++. The experiments are run on a Linux server with an Intel Xeon 6342 processor and 512GB memory. We terminate an algorithm if its running time exceeds 10^5 seconds.

Parameter settings. By default, the privacy budget ϵ is set to 2 for all datasets. We also allow ϵ to vary from 1 to 3 with increments of 0.5. For MultiR, we set $\epsilon_0 = 0.1\epsilon$, $\epsilon_1 = 0.5\epsilon$ and $\epsilon_2 = 0.4\epsilon$. For each algorithm, we run it for 10 rounds and report the average relative error.

Evaluate the effectiveness across different datasets. In Fig. 4, we report the performances of Naive, MultiR, and DBE for counting butterflies on all datasets when $\epsilon = 2$. For each dataset, we run each algorithm for 10 times and report the average relative error. We can observe that the DBE algorithm significantly outperforms Naive and MultiR under all settings. This is because, compared to Naive, DBE carefully examines the transformation between the motifs in the original graph G and those in the noisy graph G' and produces unbiased estimates. In contrast to MultiR, DBE does not require injecting substantial noise in the third round to meet privacy requirements. Note that the running time of Naive, MultiR, and DBE exceeds the time limit on the NX

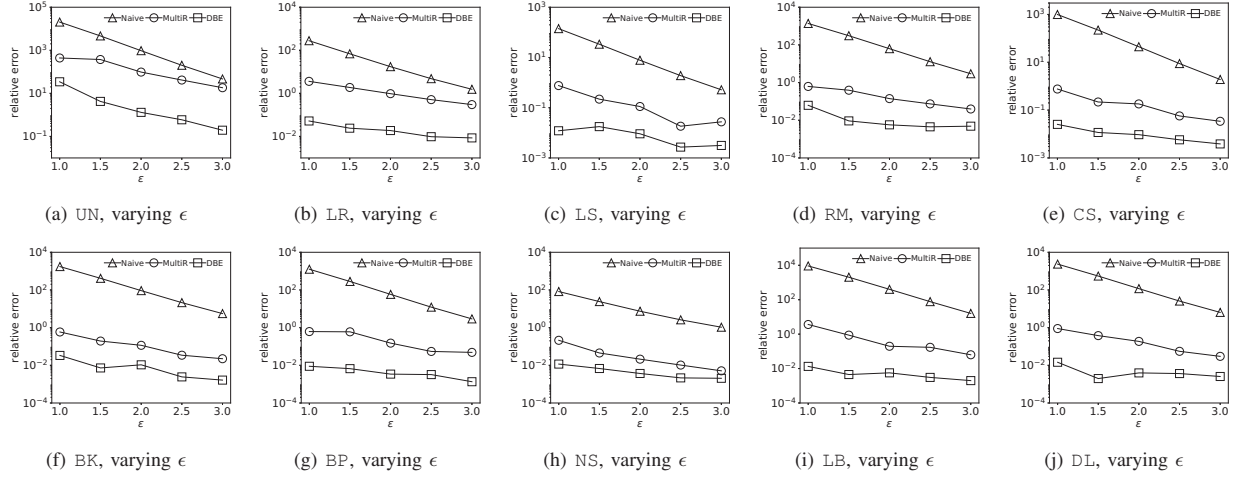


Fig. 5: Evaluate the effect of ϵ on butterfly counting

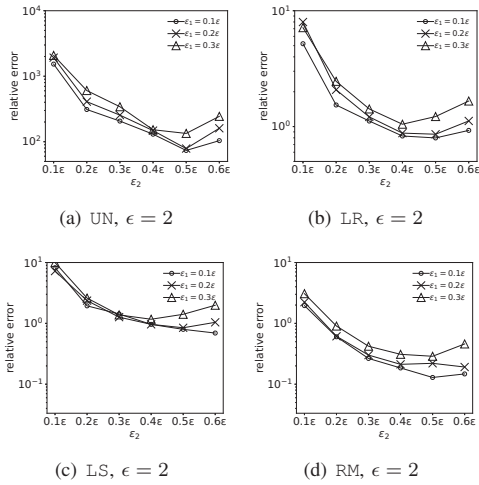


Fig. 6: Evaluate different privacy budget allocations

dataset. When we uniformly sample 10% of the noisy edges, both Naive and DBE can finish within the time limit. We can also observe that MultiR achieves much more accurate estimations than Naive. This is because Naive directly counts butterflies on the much denser noisy graph, which leads to severe over-counting. These estimations are far from being unbiased as in MultiR and DBE.

Evaluate the efficiency of butterfly counting algorithms under ϵ -edge LDP. As shown in Table IV, we report the time costs of Naive, MultiR, and DBE on 8 datasets as ϵ varies when counting butterflies. For Naive and DBE, the time cost consists of the randomized response time and the curator-side time (e.g., counting motifs on the noisy graph). For MultiR, the time cost consists of the degree estimation time (round 1), the randomized response time (round 2), and the local counting time (round 3). First, we observe that all components

of the time cost decrease as ϵ increases, except for the degree estimation time. This is because the randomized response time decreases as the flip probability $\frac{1}{1+\epsilon}$ decreases, resulting from an increasing ϵ . Also, as ϵ increases, the resulting noisy graph becomes increasingly sparse, which explains the improvement of efficiency in any procedures that involve the noisy graph. As for degree estimation, its time complexity $O(|V(G)|)$ is independent of ϵ . We also observe that Naive incurs the smallest total time cost as it requires the least server-side computation. In addition, when ϵ is larger, DBE incurs less time compared to MultiR, and vice versa. This is due to the increased density of noisy graphs on both DBE and MultiR when ϵ is small. This has more impact on DBE because it only counts motifs on the noisy graph, while MultiR considers local motifs with at least two edges in G .

Evaluate the effect of the privacy budget ϵ . As shown in Fig. 5, we report the average relative errors of Naive, MultiR, and DBE when estimating butterfly counts across 10 datasets, as ϵ varies. It is evident that all algorithms exhibit improved performance as ϵ increases. This improvement is attributed to the reduced difference between the noisy graph and the original graph G as ϵ grows. For Naive, this implies that directly counting the butterflies on the noisy graph becomes more accurate. In addition, DBE and MultiR significantly outperform Naive, and DBE outperforms MultiR by up to two orders of magnitude, as observed in previous experiments.

Evaluate the effect of privacy budget allocations on MultiR. As depicted in Fig. 6, we report the relative errors of MultiR when counting butterflies for varying privacy budget allocations across four datasets when $\epsilon = 2$. Specifically, we explore different composition of ϵ_1 , ϵ_2 , and ϵ_3 while keeping their sum constant ($\epsilon_1 + \epsilon_2 + \epsilon_3 = \epsilon$). We allow ϵ_1 and ϵ_2 to vary within the ranges of 0.1ϵ to 0.3ϵ and 0.1ϵ to 0.6ϵ , respectively. We observe that as ϵ_1 increases, there is a deterioration in overall performance. This is because a higher ϵ_1 implies a reduced budget for constructing the noisy graph and obtaining

accurate local counts in later rounds. In this way, there will be a larger difference between the original graph and the noisy graph (round 2) and more noise will be injected into local counts (round 3). We can also observe that: initially, the relative errors of `MultiR` decrease as ϵ_2 increases, but they start to rise as ϵ_2 increases further. This is because there is a trade-off between injecting less noise into the noisy graph and injecting less noise into the local motif counts. When either ϵ_2 or ϵ_3 is too small, the overall performance of `MultiR` suffers. We can see that $\epsilon_1 = 0.1\epsilon$, $\epsilon_2 = 0.5\epsilon$, and $\epsilon_3 = 0.4\epsilon$ offer a balanced budget allocation that generally benefits `MultiR`. Thus, we adopt this privacy budget allocation for `MultiR`.

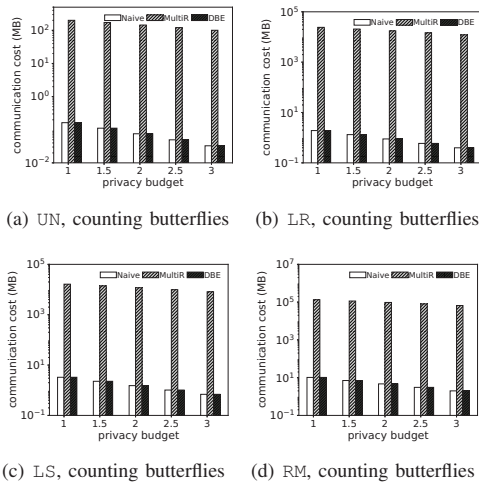


Fig. 7: The communication cost of Naive, `MultiR`, and DBE

Evaluate the communication cost of butterfly counting algorithms under ϵ -edge LDP. In practice, all algorithms under ϵ -edge LDP need to communicate with the data curator. In Fig 7, we report the message sizes (in MB) in each algorithm on two datasets, as ϵ varies. We observe that Naive and DBE require approximately the same message sizes. This is because given a fixed ϵ , Naive and DBE apply randomized responses with the same flipping probability, leading to the same number of noisy edges in the expected value. It can also be observed that `MultiR` incurs noticeably higher communication costs compared to Naive and DBE. This is because `MultiR` needs to send the degree estimates from round 1, the noisy edges from round 2, and the local counts in round 3. Additionally, `MultiR` needs to download the noisy graph for all vertices. In summary, DBE incurs the least communication costs while yielding the most accurate butterfly counts among these butterfly counting algorithms with edge LDP.

VI. RELATED WORK

Here we review the related works on butterfly counting on bipartite graphs and motif counting under differential privacy. **Butterfly counting on bipartite graphs.** The butterfly is considered the smallest unit of cohesion on bipartite graphs [4], [5], [6], [8], and butterfly counting has been studied

extensively. The first efficient exact algorithm for butterfly counting is proposed in [5], which outperforms generic matrix-multiplication-based methods [43]. Sanei et al. [4] introduce an improved exact algorithm along with an approximate solution with improved efficiency. Wang et al. [6] present a butterfly counting algorithm based on vertex priority that is scalable to billion-scale graphs. Variants of butterfly counting have received much attention in recent years, including butterfly counting in uncertain graphs [7], I/O efficient butterfly counting [12], batch-dynamic butterfly counting [12], GPU-based butterfly counting [10], butterfly approximation in streaming graphs [18], [44] and parallel butterfly counting [45].

Motif counting under differential privacy. Motif counting under *central differential privacy* has received much attention for various motifs, including triangles, k-stars, and k-triangles [28], [46], [40], [47]. However, the techniques in these works are not suitable for motif counting under edge LDP due to the data curator's limited access to the entire graph [29], [33], [48], [38], [39], [49]. For triangle counting under edge LDP, [29] introduces one-round and two-round algorithms, and [33] further enhances communication efficiency and reduces estimation error. [48] provides a detailed technical analysis of these algorithms. In this paper, the `MultiR` algorithm is inspired by the two-round triangle counting algorithm under edge LDP, which introduces significant noise due to the high global sensitivity of butterfly counts. Our DBE algorithm improves data utility by minimizing excessive communication between vertices and the data curator, thereby addressing the challenge posed by high global sensitivity. [38] and [39] explore triangle counting in a localized setting, where each vertex can observe its two-hop neighborhood. While such a formulation simplifies reporting local triangle counts, it requires a stronger variant of LDP. A recent work [49] estimates the 4-cycle and triangle counts under the shuffle model, where users' messages are shuffled before being sent to the data curator. This model assumes that the data curator and the shuffler do not collude.

VII. CONCLUSION

In this paper, we study butterfly counting on bipartite graphs under edge LDP. To address the "dense noisy graph" issue of Naive, we propose a multi-round algorithm `MultiR` to provide unbiased estimates of butterfly counts. For better data utility, we propose the DBE algorithm, which directly utilizes the motif counts on the noisy graph. DBE achieves significant accuracy improvements as it avoids injecting substantial noise from high global sensitivity by reducing vertex-to-data-curator communication. Experiments on real-world bipartite graphs validate the effectiveness and the efficiency of DBE.

VIII. ACKNOWLEDGMENT

Kai Wang is supported by NSFC 62302294 and NSFC U2241211. Wenjie Zhang is supported by ARC DP230101445 and FT210100303. Xuemin Lin is supported by NSFC U2241211, NSFC U20B2046, and 23H020101910. Ying Zhang is supported by ARC DP210101393, LP210301046, and DP230101445. Kai Wang is the corresponding author.

REFERENCES

- [1] Z. Li, X. Shen, Y. Jiao, X. Pan, P. Zou, X. Meng, C. Yao, and J. Bu, "Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1677–1688.
- [2] X. Chen, K. Wang, X. Lin, W. Zhang, L. Qin, and Y. Zhang, "Efficiently answering reachability and path queries on temporal bipartite graphs," *Proceedings of the VLDB Endowment*, 2021.
- [3] C. M. O'Connor, J. U. Adams, and J. Fairman, "Essentials of cell biology," *Cambridge, MA: NPG Education*, vol. 1, p. 54, 2010.
- [4] S.-V. Sanehi-Mehri, A. E. Sariyuce, and S. Tirthapura, "Butterfly counting in bipartite networks," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 2150–2159.
- [5] J. Wang, A. W.-C. Fu, and J. Cheng, "Rectangle counting in large bipartite graphs," in *2014 IEEE International Congress on Big Data*. IEEE, 2014, pp. 17–24.
- [6] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Vertex priority based butterfly counting for large-scale bipartite networks," *Proceedings of the VLDB Endowment*, vol. 12, no. 10, p. 1139–1152, 2019.
- [7] A. Zhou, Y. Wang, and L. Chen, "Butterfly counting on uncertain bipartite graphs," *Proceedings of the VLDB Endowment*, vol. 15, no. 2, pp. 211–223, 2021.
- [8] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Efficient bitruss decomposition for large-scale bipartite graphs," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 661–672.
- [9] A. Abidi, L. Chen, R. Zhou, and C. Liu, "Searching personalized k-wing in bipartite graphs," *IEEE Transactions on Knowledge and Data Engineering*, 2022.
- [10] Q. Xu, F. Zhang, Z. Yao, L. Lu, X. Du, D. Deng, and B. He, "Efficient load-balanced butterfly counting on gpu," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2450–2462, 2022.
- [11] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, "Efficient maximal biclique enumeration for large sparse bipartite graphs," *Proceedings of the VLDB Endowment*, vol. 15, no. 8, pp. 1559–1571, 2022.
- [12] Z. Wang, L. Lai, Y. Liu, B. Shui, C. Tian, and S. Zhong, "I/o-efficient butterfly counting at scale," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–27, 2023.
- [13] D. Gibson, R. Kumar, and A. Tomkins, "Discovering large dense subgraphs in massive graphs," in *Proceedings of the 31st international conference on Very large data bases*, 2005, pp. 721–732.
- [14] X. Su and T. M. Khoshgoftaar, "A survey of collaborative filtering techniques," *Advances in artificial intelligence*, vol. 2009, 2009.
- [15] S. G. Aksoy, T. G. Kolda, and A. Pinar, "Measuring and modeling bipartite graphs with community structure," *Journal of Complex Networks*, vol. 5, no. 4, pp. 581–603, 2017.
- [16] Z. Huang, "Link prediction based on graph topology: The predictive value of generalized clustering coefficient," *Available at SSRN 1634014*, 2010.
- [17] Z. Huang, X. Li, and H. Chen, "Link prediction approach to collaborative filtering," in *Proceedings of the 5th ACM/IEEE-CS joint conference on Digital libraries*, 2005, pp. 141–142.
- [18] R. Li, P. Wang, P. Jia, X. Zhang, J. Zhao, J. Tao, Y. Yuan, and X. Guan, "Approximately counting butterflies in large bipartite graph streams," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 12, pp. 5621–5635, 2021.
- [19] B. Lyu, L. Qin, X. Lin, Y. Zhang, Z. Qian, and J. Zhou, "Maximum biclique search at billion scale," *Proceedings of the VLDB Endowment*, 2020.
- [20] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 425–438.
- [21] Y. Zhang, J. Wei, X. Zhang, X. Hu, and W. Liu, "A two-phase algorithm for generating synthetic graph under local differential privacy," in *Proceedings of the 8th International Conference on Communication and Network Security*, 2018, pp. 84–89.
- [22] Q. Ye, H. Hu, M. H. Au, X. Meng, and X. Xiao, "Lf-gdpr: A framework for estimating graph metrics with local differential privacy," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 4905–4920, 2020.
- [23] M. Hay, V. Rastogi, G. Miklau, and D. Suciu, "Boosting the accuracy of differentially-private histograms through consistency," *arXiv preprint arXiv:0904.0942*, 2009.
- [24] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate estimation of the degree distribution of private networks," in *2009 Ninth IEEE International Conference on Data Mining*. IEEE, 2009, pp. 169–178.
- [25] W.-Y. Day, N. Li, and M. Lyu, "Publishing graph degree distribution with node differential privacy," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 123–138.
- [26] X. Ding, X. Zhang, Z. Bao, and H. Jin, "Privacy-preserving triangle counting in large graphs," in *Proceedings of the 27th ACM international conference on information and knowledge management*, 2018, pp. 1283–1292.
- [27] K. Nissim, S. Raskhodnikova, and A. Smith, "Smooth sensitivity and sampling in private data analysis," in *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, 2007, pp. 75–84.
- [28] V. Karwa, S. Raskhodnikova, A. Smith, and G. Yaroslavtsev, "Private analysis of graph structure," *Proceedings of the VLDB Endowment*, vol. 4, no. 11, pp. 1146–1157, 2011.
- [29] J. Imola, T. Murakami, and K. Chaudhuri, "Locally differentially private analysis of graph statistics," in *USENIX Security Symposium*, 2021, pp. 983–1000.
- [30] H. Jiang, J. Pei, D. Yu, J. Yu, B. Gong, and X. Cheng, "Applications of differential privacy in social network analysis: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 108–127, 2021.
- [31] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, 2014, pp. 1311–1322.
- [32] J. Yang, Y. Peng, D. Ouyang, W. Zhang, X. Lin, and X. Zhao, "(p, q)-biclique counting and enumeration for large sparse bipartite graphs," *The VLDB Journal*, pp. 1–25, 2023.
- [33] J. Imola, T. Murakami, and K. Chaudhuri, "Communication-efficient triangle counting under local differential privacy," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 537–554.
- [34] L. Dhulipala, Q. C. Liu, S. Raskhodnikova, J. Shi, J. Shun, and S. Yu, "Differential privacy from locally adjustable graph algorithms: k-core decomposition, low out-degree ordering, and densest subgraphs," in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2022, pp. 754–765.
- [35] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," *Journal of the American Statistical Association*, vol. 60, no. 309, pp. 63–69, 1965.
- [36] C. Dwork, A. Roth *et al.*, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [37] Z. Qin, T. Yu, Y. Yang, I. Khalil, X. Xiao, and K. Ren, "Generating synthetic decentralized social graphs with local differential privacy," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 425–438.
- [38] Y. Liu, S. Zhao, Y. Liu, D. Zhao, H. Chen, and C. Li, "Collecting triangle counts with edge relationship local differential privacy," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 2008–2020.
- [39] H. Sun, X. Xiao, I. Khalil, Y. Yang, Z. Qin, H. Wang, and T. Yu, "Analyzing subgraph statistics from extended local views with decentralized differential privacy," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 703–717.
- [40] X. Ding, S. Sheng, H. Zhou, X. Zhang, Z. Bao, P. Zhou, and H. Jin, "Differentially private triangle counting in large graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 11, pp. 5278–5292, 2021.
- [41] C. S. Calude, M. J. Dinneen, and R. Hua, "Qubo formulations for the graph isomorphism problem and related problems," *Theoretical Computer Science*, vol. 701, pp. 54–69, 2017.
- [42] N. Chiba and T. Nishizeki, "Arboricity and subgraph listing algorithms," *SIAM Journal on computing*, vol. 14, no. 1, pp. 210–223, 1985.
- [43] N. Alon, R. Yuster, and U. Zwick, "Finding and counting given length cycles," *Algorithmica*, vol. 17, no. 3, pp. 209–223, 1997.
- [44] A. Sheshbolouki and M. T. Özsu, "sgrapp: Butterfly approximation in streaming graphs," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 16, no. 4, pp. 1–43, 2022.

- [45] J. Shi and J. Shun, "Parallel algorithms for butterfly computations," in *Massive Graph Analytics*. Chapman and Hall/CRC, 2022, pp. 287–330.
- [46] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "Private release of graph statistics using ladder functions," in *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, 2015, pp. 731–745.
- [47] T. Lv, H. Li, Z. Tang, F. Fu, J. Cao, and J. Zhang, "Publishing triangle counting histogram in social networks based on differential privacy," *Security and Communication Networks*, vol. 2021, pp. 1–16, 2021.
- [48] T. Eden, Q. C. Liu, S. Raskhodnikova, and A. Smith, "Triangle counting with local edge differential privacy," *arXiv preprint arXiv:2305.02263*, 2023.
- [49] J. Imola, T. Murakami, and K. Chaudhuri, "Differentially private triangle and 4-cycle counting in the shuffle model," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1505–1519.